FDDT documentation

FDDT — Fraud Document Detection Tool authenticates third-party documents (vendor invoices, school documents, quotations, travel claims and the payment evidence behind them) *before* an approval or payment is made. A user submits a **case** of PDF documents; the system extracts their data, runs a battery of automated checks, turns the results into an **explainable** risk score, and hands the case to a human reviewer (Reviewer L1, with escalation routing a case to Reviewer L2). Every step is written to an append-only audit log. Several client companies share one deployment, each isolated from the others by the application and by PostgreSQL Row-Level Security.

These docs describe the system **as implemented**, checked against the code. The capabilities included in the current build, and those outside its scope, are collected in the table in architecture/overview.md.

Start here

If you want to…	Read
Learn to use the app as a Company User (Submitter, Reviewer L1, Reviewer L2)	COMPANY_USER_MANUAL.md
Manage the multi-tenant system as a Platform Administrator	PLATFORM_ADMIN_MANUAL.md
Understand the whole system in ten minutes	architecture/overview.md
Understand companies, roles and tenant isolation (RLS)	multi-tenancy.md
Tune processing queues, worker pools and Azure rate limits	processing-queues.md
Follow one case from upload to report	architecture/data-flow.md
Run it on your machine	deployment/local-setup.md
Know what to configure and where to get keys	deployment/environment-variables.md
Understand what each check detects and its limits	the pipeline docs below
Call or change the API	api/README.md → Swagger at `http://localhost:8000/docs`
Change the database	database/schema.md, database/erd.md
Work on the UI	frontend/screens.md, frontend/components.md
Deploy to production on Azure (architecture, service tiers, costs, step-by-step runbook)	deployment/production-deployment.md
See the remaining pre-production gaps	deployment/azure-migration-notes.md
See how the Azure demo is deployed	deployment/azure-demo-deployment.md
Read what changed in the multi-tenancy / queues work, and why	MULTI_TENANCY_CHANGE_REPORT.md

Contents

User Manuals

- COMPANY_USER_MANUAL.md — Comprehensive guide for client organizations covering all 3 company roles (User/Submitter, Reviewer L1, Reviewer L2), case submission, bulk intake, review workflows, automated forensic checks, overlays, and company settings. Includes high-resolution interface screenshots.

- PLATFORM_ADMIN_MANUAL.md — Complete operational manual for Platform Administrators covering tenant provisioning, company limits, user administration across all 4 roles, password resets, rule templates, queue monitoring, usage analytics, and immutable audit history. Includes high-resolution interface screenshots.
- user-manual.md — Consolidated role-by-role overview guide.

Architecture

- overview.md — system diagram, tech stack, the key design decisions and their rationale, the scope-of-the-current-build table, security posture.
- data-flow.md — sequence diagram and step-by-step walkthrough, Celery task names and queues.
- multi-tenancy.md — companies, roles, two-layer tenant isolation (application + RLS), platform-admin support access, blob layout, billing & usage.
- processing-queues.md — the three queues, fair-share scheduling, idempotent writes, calibrated Azure rate limits, database connections and PgBouncer, load tests.

Pipeline — what each stage does

Each page covers what it detects, the algorithm and libraries, the real input/output shape, known limitations and the risk rules it feeds.

1. Upload & intake · 1a. Bulk upload (a zip of cases)
2. Extraction & normalization
3. Field validation
4. Cross-document consistency
5. Issuer verification
6. Metadata forensics
7. ELA & copy-move
8. Visual review & AI-generation
9. Signature verification
10. Duplicate detection
11. Risk scoring engine

Reference

- API + `openapi.json` (generated from the code)
- Database schema and ERD (generated from the live database by `tools/gen_db_docs.py`)
- Frontend screens and components
- Per-case forensic report

Deployment

- Local setup · Environment variables
- Production deployment — Azure Container Apps architecture, S/M/L service tiers with monthly costs (UAE North list prices), deployment runbook, operations and go-live checklist
- Azure migration notes — what is Azure-ready and the remaining gaps
- Azure demo deployment — the low-cost demo topology and operating notes

Change records

- MULTI_TENANCY_CHANGE_REPORT.md — every change, decision and problem in the multi-tenancy, processing-queue and follow-up work

Ground rules the design follows

These principles explain many choices you will meet in the code:

- **No model is trained or hosted.** Intelligence is a managed cloud API or a classical library.
- **A transparent rules engine, not a model, produces the risk score**, with versioned, admin-tunable rules.
- **One generic, template-free extraction pipeline** for every document type.
- **Original files are immutable** and hash-verified; everything else derives from them.
- **The audit log is append-only.**
- **Weak signals are labelled weak** — signature comparison, AI-generation detection and ELA are advisory, never verdicts, and never described as "verified".
- **The UI is English-only**; only extracted Arabic values need right-to-left handling.

Keeping these docs true

- The **OpenAPI snapshot** and the **database docs** are generated — regenerate them rather than editing: `api/openapi.json` (command in api/README.md); `database/*.md` with `cd backend && .venv/Scripts/python ../docs/tools/gen_db_docs.py` against the migrated local Postgres — re-run after every migration.
- The **pipeline docs** quote real constants, thresholds and result shapes. When you change an algorithm or a default rule, update its page and the rule tables (the default rules live in `backend/app/services/risk_rule_seed.py` and the platform's `risk_rule_templates`).
- When you add or change a capability listed in the overview's scope table, update that table.
- The **PDFs in `docs-pdf/`** are built from these Markdown files (and `api/openapi.json`) — never edit them by hand. After changing any doc, rebuild: `cd docs/tools && npm install && npm run build` (see tools/README.md).